WALCHAND COLLEGE OF ENGINEERING, SANGLI

(An Autonomous Institute, Sangli, Maharashtra)

ROBOTICS LABORATORY

EXPERIMENT LA2

Dobot Magician 3D Robot Simulation and Control

Submitted by	Atharv Kulkarni
PRN	245109074
Programme / Department	B.Tech. – Computer Science and Engineering
Class / Year	Third Year (TY B.Tech.)
Course / Subject	Robotics Laboratory

Project Repository and Live Demonstration

GitHub Repository: https://github.com/atharv853/Dubot_Simulation

Live Application: https://dubot-simulation.vercel.app/

EXPERIMENT LA2

Experiment Title:

Design and implementation of a web-based 3D Dobot Magician robot simulation with motion controls, workcell objects, teaching sequences, block-based programming, and real-time controller synchronisation.

AIM:

To develop and study an interactive web-based simulation of a Dobot robotic arm, enabling users to visualise robot motion, control its operation, arrange objects in a virtual workcell, create teaching sequences, and explore program logic without requiring a physical robot for basic simulation activities.

OBJECTIVES:

- To understand how a robotic arm can be represented and visualised in a three-dimensional virtual environment.
- To provide an interactive viewport for observing the robot model, worktable, scene objects, placement preview, reach envelope, and motion trajectory.
- To implement operation controls and maintain simulation state as controls and object properties are changed.
- To support teaching sequences and block-based program construction through a Blockly-style panel.
- To study robot kinematics and coordinate-frame conversion as part of motion and pose calculation.
- To provide login/session handling and route protection for the application.
- To explore communication between a host simulator and a controller interface through a real-time messaging layer.
- To deploy the application as a web-accessible demonstration and document its major features with screenshots.

THEORY:

A robot simulator is a software environment used to represent a robot, its workspace, and selected actions digitally. It allows learners to inspect robot configuration and experiment with motion commands in a controlled setting. A 3D simulator is especially useful because the user can view the robot and its surrounding objects spatially rather than relying only on numerical coordinates.

This project, Dubot Simulation, is organised around a browser-based user interface, a 3D simulation layer, state and kinematics utilities, identity/access components, and real-time controller synchronisation. The application uses Next.js and React with TypeScript for its web interface. Three.js, React Three Fiber, and Drei support the 3D scene. Zustand is used for shared simulation state, while Socket.IO and a Node.js server support real-time communication. Tailwind CSS is included for interface styling. These technologies are identified from the project repository.

1. Three-dimensional robot visualisation

The viewport displays the robot model together with the virtual table/environment and scene objects. Additional visual elements, such as a placement preview, reach envelope, and trajectory line, help the user understand possible placements and the path associated with robot movement.

2. Robot state and kinematics

Simulation controls modify the application state. The kinematics module calculates robot poses from the current state, while coordinate utilities support conversion between coordinate frames. These calculations provide the basis for consistent visualisation and movement within the virtual workcell.

3. Teaching sequences and block-based programming

The teaching panel supports working with sequences of actions. The block-programming panel provides a visual way to organise program logic using blocks, making it easier to understand and communicate the order of robot operations than writing every instruction as text.

4. Real-time controller communication

The project contains host-side and controller-side synchronisation components, a Socket.IO client, command and state endpoints, and a server-side relay. Together, these components are intended to pass control commands and state information between the simulator and controller interface. Actual synchronisation should be demonstrated and captured during testing.

5. Authentication and access control

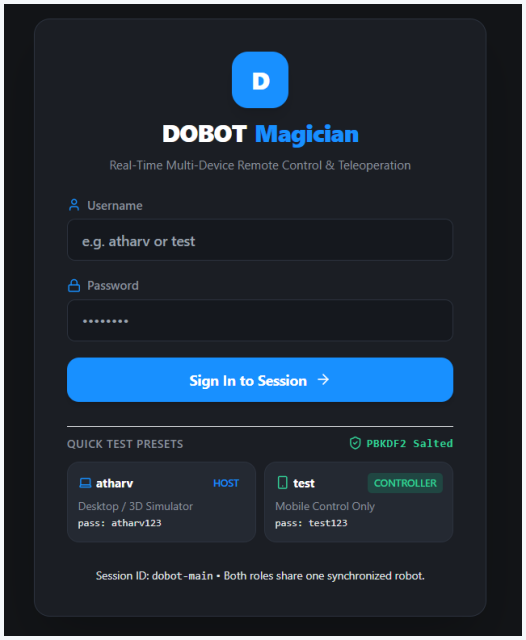
The source includes login and logout endpoints, session-token utilities, and middleware/edge authentication logic. These components are used to manage user sessions and protect relevant application routes.

EXPERIMENT STEPS WITH SCREENSHOT OF EACH STEP:

Perform the following steps in the deployed application. Insert a genuine screenshot at each marked location before submission. The browser layout and available controls may vary with the current deployed version.

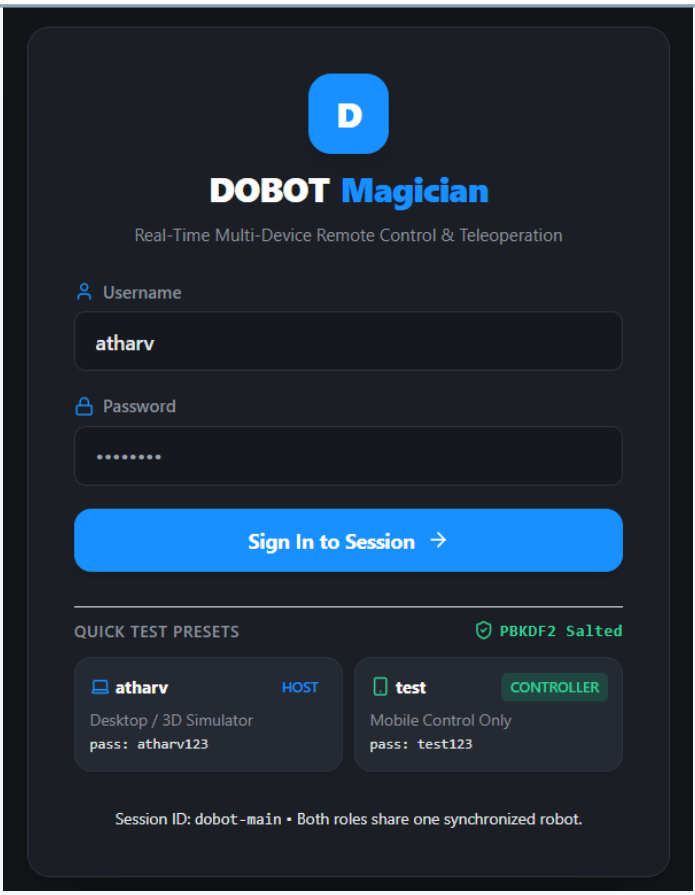
Step 1 – Open the application

Open the live application URL in a modern web browser. Confirm that the page loads and identify the available sign-in or entry screen.



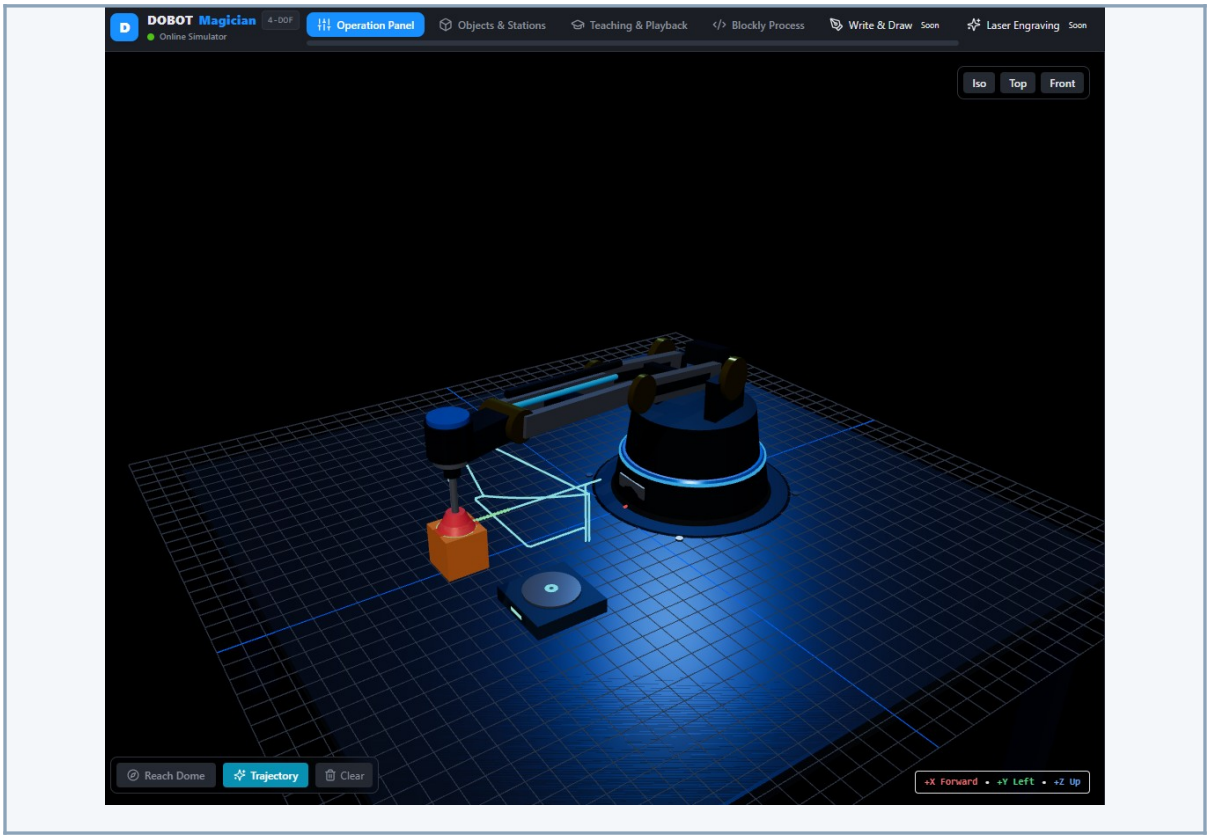
Step 2 – Sign in

Enter the credentials required by the application and submit the login form. Do not include passwords or other secrets in the report screenshot.



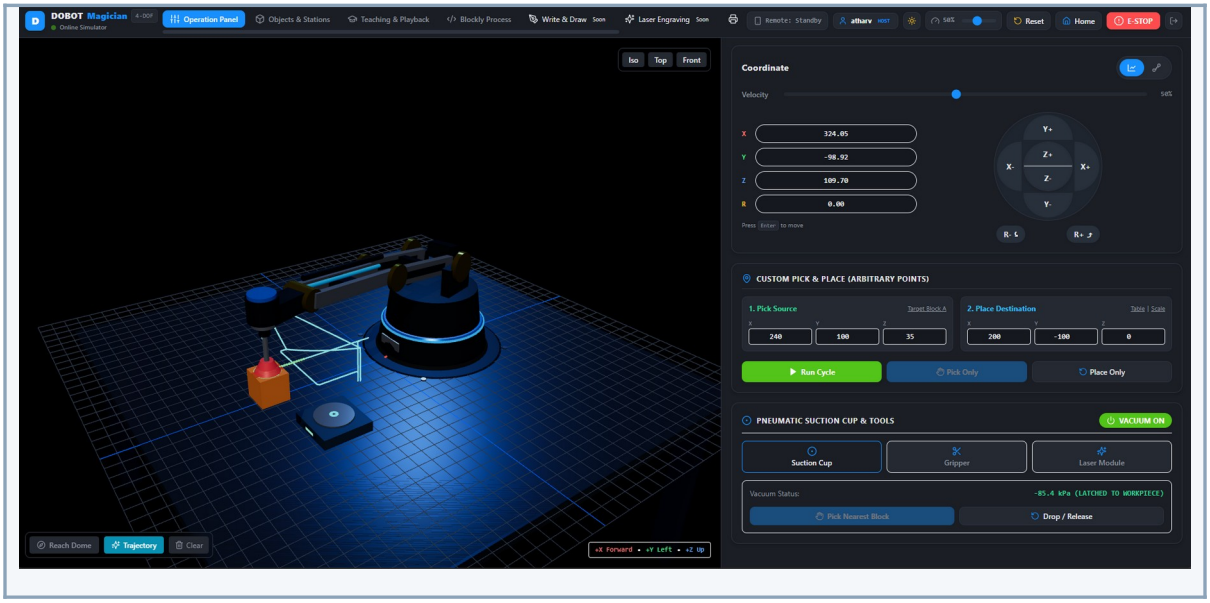
Step 3 – Inspect the simulator workspace

Open the simulator page and identify the 3D viewport, navigation bar, robot-operation panel, workcell-object panel, teaching panel, block-programming panel, and status information.



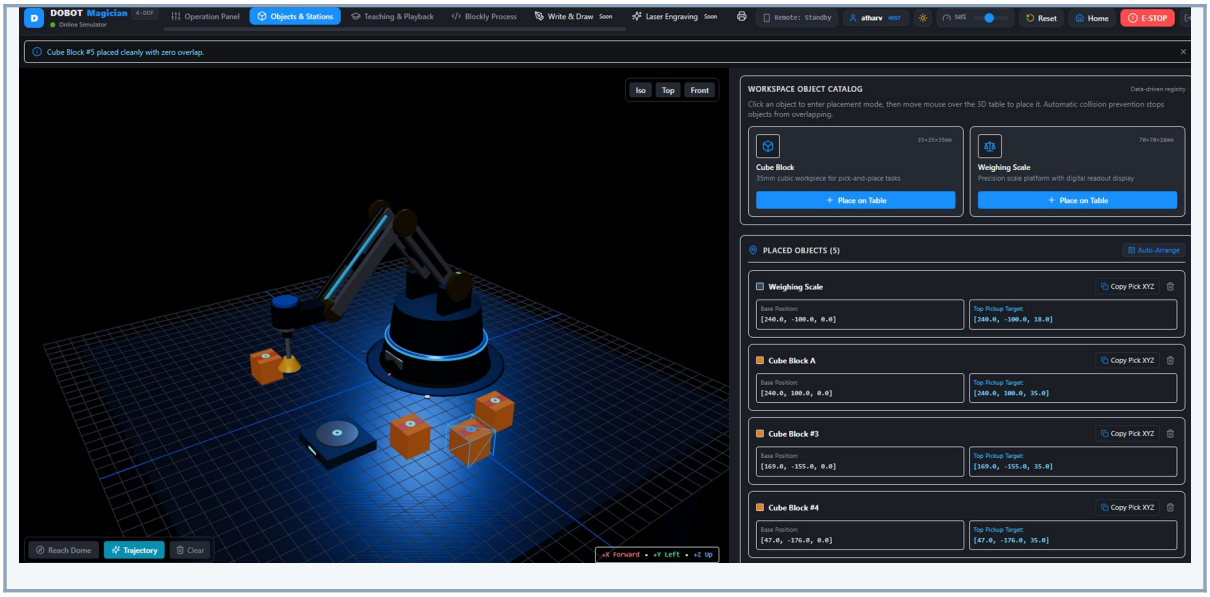
Step 4 – Explore robot operation controls

Use the available operation controls to inspect supported movement or pose settings. Observe how the robot model and status indicators respond.



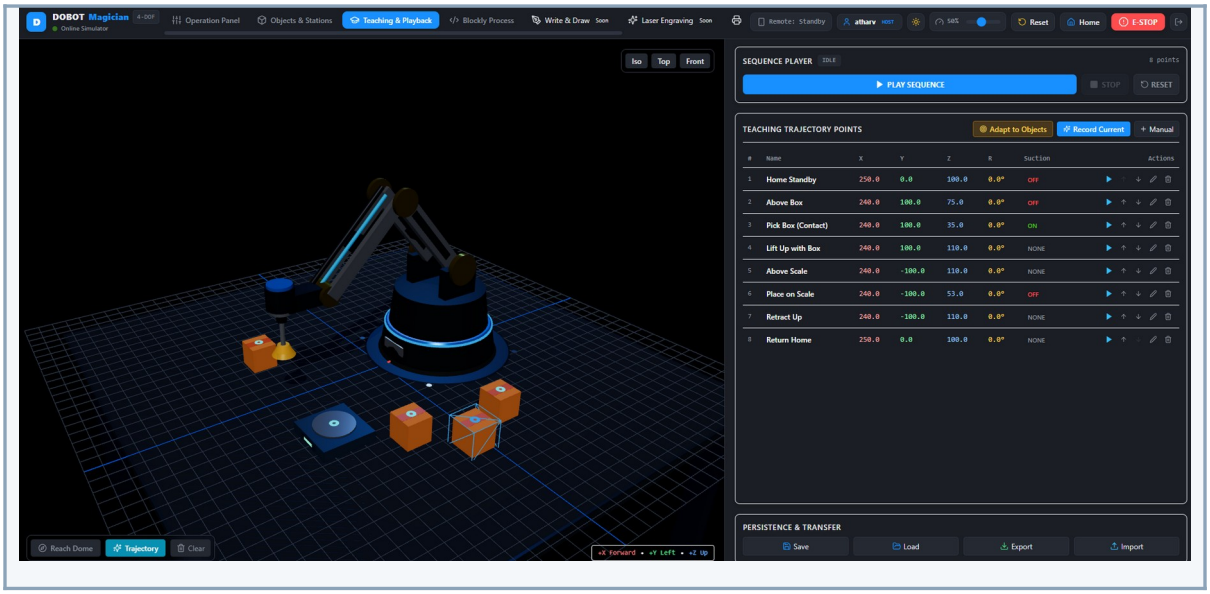
Step 5 – Arrange workcell objects

Use the object panel to add, select, position, or edit available virtual objects. Observe their representation in the 3D scene.



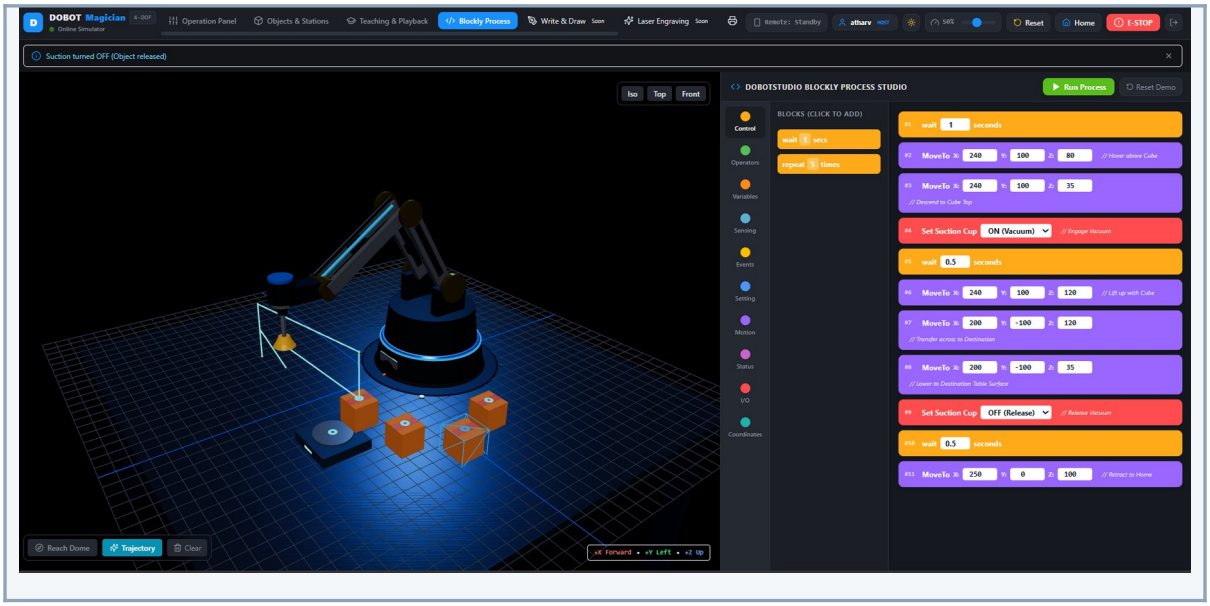
Step 6 – Inspect teaching sequences

Open the teaching panel and examine the available sequence controls. Create or play a short sequence if the interface supports it, and record the resulting behaviour.



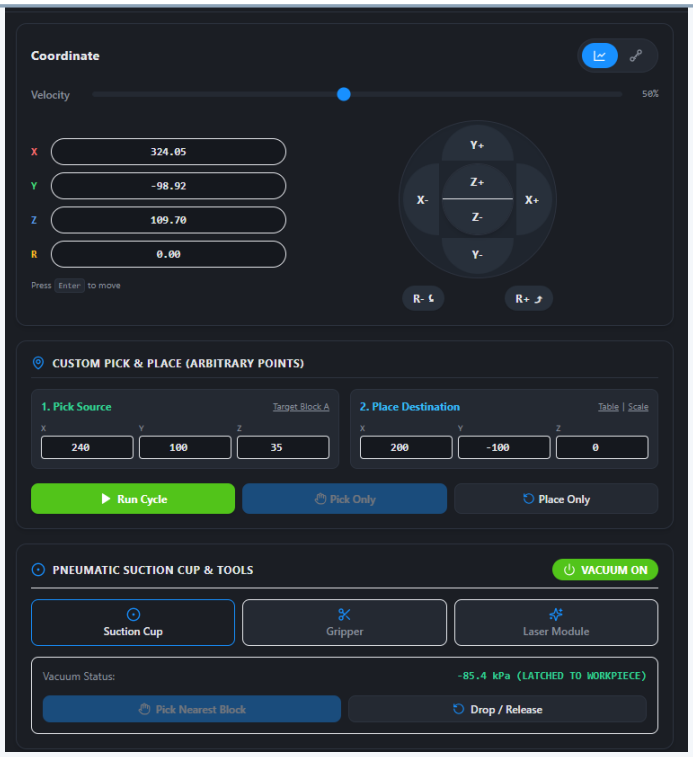
Step 7 – Explore block-based programming

Open the Blockly panel and inspect how visual blocks can be arranged to express a sequence of operations. Capture the program arrangement and its available run/preview controls.



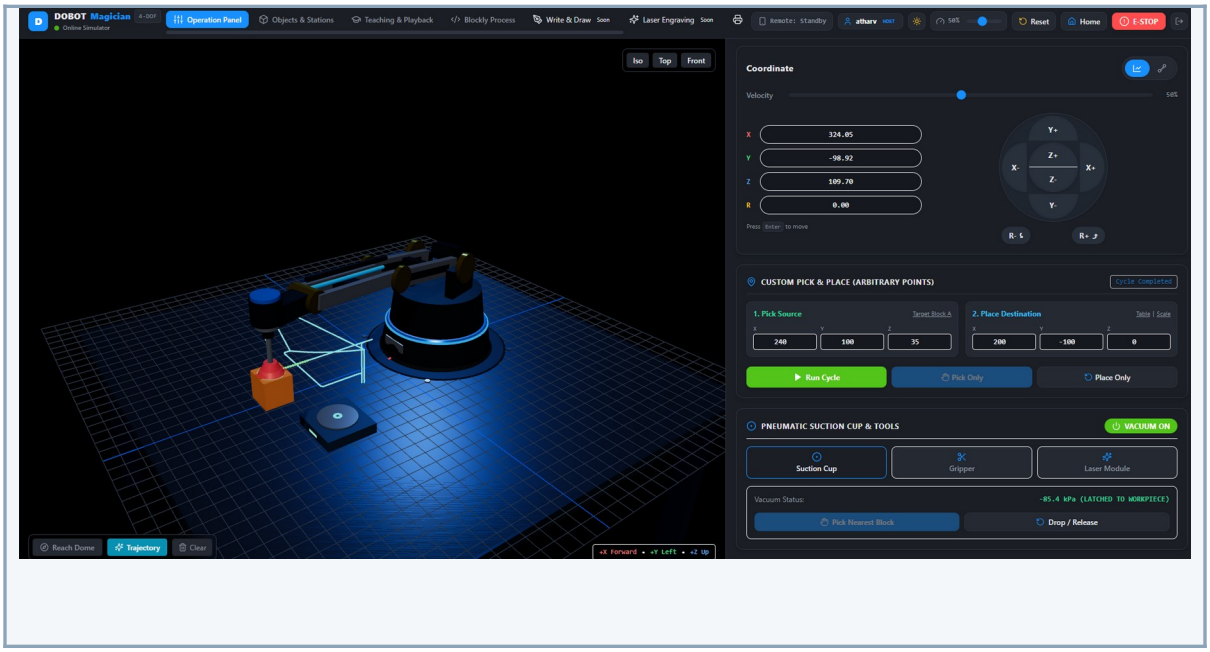
Step 8 – Open the controller interface

Navigate to the controller page, if available. Inspect connection/synchronisation status and test only the commands supported by the deployed instance.



Step 9 – Observe the result

Record the resulting robot pose, scene changes, motion trail, status message, or other visible output. Capture the final state as evidence of the experiment.



IMPLEMENTATION WITH OUTPUT:

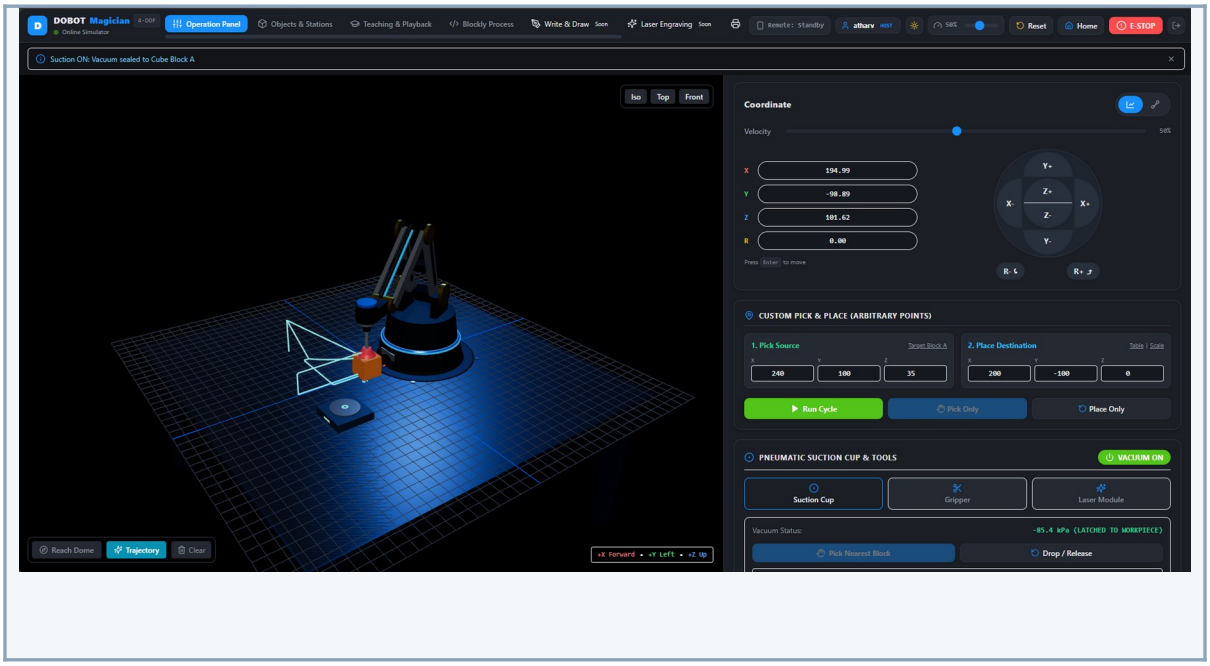
The application is implemented as a TypeScript-based web project. Its main implementation areas are summarised below:

Implementation area	Description
User interface	Next.js / React pages and reusable interface components for login, simulator navigation, operation controls, object management, teaching, status display, and Blockly-based programs.
3D simulation	React Three Fiber, Drei, and Three.js components render the Dobot model, virtual environment, scene objects, reach envelope, placement preview, and trajectory line.
Simulation state	A shared Zustand store keeps the simulator state used by interface panels and viewport components.
Kinematics and coordinates	Dedicated TypeScript utilities calculate robot poses and support coordinate-frame conversion.
Real-time communication	Socket.IO client/server components, synchronisation hooks, and API endpoints are included for command forwarding and state retrieval.
Authentication	Login/logout routes, token utilities, and middleware/edge-auth modules support session management and access checks.
Build and testing tools	The package configuration includes Next.js scripts and Vitest-based tests for kinematics and real-time simulation logic.

Expected / observed output

The expected output is an interactive browser-based robot simulation in which the user can view the 3D robot and workcell, interact with supported controls, manage virtual objects, inspect teaching sequences, and construct block-based programs. The interface should display resulting simulation state and visual movement where the corresponding actions are supported. The controller page and real-time features should be reported according to what is actually observed during the test.

Output evidence: insert your final simulator screenshot below. This report does not claim specific measured timings, accuracy values, or successful hardware motion because such measurements were not provided.



PROJECT LINKS:

Source code repository: https://github.com/atharv853/Dubot_Simulation

Deployed web application: https://dubot-simulation.vercel.app/

CONCLUSION:

The Dubot Simulation project provides a web-based environment for studying a Dobot-style robotic arm through 3D visualisation and interactive simulation controls. The codebase separates the user interface, virtual scene, simulation state, kinematics, authentication, and real-time communication into dedicated modules. The project demonstrates how modern web technologies can be used to build an accessible robotics learning interface without requiring the user to begin with a physical robot. Completing the experiment also provides practical exposure to robot pose representation, coordinate frames, teaching sequences, visual programming, and client-server communication. The final report should be accompanied by screenshots captured from the actual deployed application, including the results observed during testing.